

Concept 2: RESTful API Design

REST is a set of rules for how your front-end and back-end talk to each other over HTTP. It's simple once you see the pattern.

You have **resources** (the things — like RFQ, Reminder). You have **HTTP methods** (the actions — what you want to do to that thing). That's it.

There are 4 main methods, and they map directly to CRUD:

- **POST** = Create → "Make a new one"
- **GET** = Read → "Give me one" or "Give me all of them"
- **PUT** = Update → "Replace this one with new data"
- **DELETE** = Delete → "Remove this one"

Now combine them with a resource URI, using your bosses' naming convention:

POST	/rfq-manager/v1/rfqs	→ Create a new RFQ
GET	/rfq-manager/v1/rfqs	→ List all RFQs
GET	/rfq-manager/v1/rfqs/42	→ Get RFQ number 42
PUT	/rfq-manager/v1/rfqs/42	→ Update RFQ number 42
DELETE	/rfq-manager/v1/rfqs/42	→ Delete RFQ number 42

Notice the pattern: you never put verbs in the URL. You don't write `/rfq-manager/v1/createRfq`. The **method** (POST, GET, PUT, DELETE) is the verb. The **URI** is just the noun.

Sometimes you need a **custom action** that doesn't fit CRUD cleanly. For example, submitting an RFQ for review. That looks like:

```
POST /rfq-manager/v1/rfqs/42/submit
```

It's still a POST (you're creating a submission), and the action goes at the end of the path.

One more thing: resource names are always **plural** and **lowercase**. It's `/rfqs` not `/rfq`, `/reminders` not `/Reminder`.

When we talk about REST, endpoints, URIs, HTTP methods, CRUD — **all of that is only about API resources**. The stuff you expose to the front-end.

Everything else (intermediate entities, internal logic, automatic stage creation, etc.) is just regular code inside your service doing database operations. No REST involved.

2. RESTful API Design — CRUD & URI Conventions

REST = Representational State Transfer. Your API exposes **resources** (nouns, not verbs) and uses **HTTP methods** to act on them.

HTTP Method	Action	Example
GET	Read	GET /rfqs → list all RFQs
POST	Create	POST /rfqs → create a new RFQ
PATCH	Partial Update	PATCH /rfqs/{id} → update some fields
PUT	Full Replace	PUT /rfqs/{id} → replace entire resource
DELETE	Remove	DELETE /rfqs/{id} → remove resource

URI Naming Convention at BACAB:

```
{microservice_name}/{version}/{resource_name}/{id_or_custom_action}
```

Meeting link: "Le préfixe en premier est le nom du microservice, pas juste 'api' — car chaque MS a son API."

So instead of:

```
/api/v1/rfqs ❌ (ambiguous — which microservice?)
```

Use:

```
/rfq-manager/v1/rfqs ✅ (clear ownership)
```

Terms to know:

- **URL** = full address: `https://api.example.com:8080/rfq-manager/v1/rfqs/123`
- **URI** = the path after IP:port: `/rfq-manager/v1/rfqs/123`
- **Resource name** in the URI must match consistently across all endpoints